# System Level Energy and Cost Characterization of Hybrid Cloud Workloads

Waylon Luo
*Kent State University*
Kent, OH, USA

Joao Vitor Macambira Donaton
*Kent State University*
Kent, OH, USA

Priyabrata Senapati
*Kent State University*
Kent, OH, USA

Qiang Guan
*Kent State University*
Kent, OH, USA

***Abstract*—Near-term quantum workloads depend on hybrid quantum-classical cloud architectures, yet their power and performance dynamics remain difficult to characterize. To address this, we present a high-level simulation framework to evaluate power, energy, and cost across heterogeneous QPU and CPU clouds. By modeling macro level interactions such as job arrivals, workflow iterations, and device power states, the framework orchestrates administrative-level evaluation without requiring detailed quantum properties. To demonstrate the framework with selected configurations and workload, we sweep the variational iteration count over a fixed fleet, holding circuit width, depth, shots, and arrival patterns constant, so that loop length serves as the sole independent variable. While computation and per-job energy scale linearly, QPU occupancy and turnaround depart abruptly from linearity beyond a critical iteration count: past it, nearly all quantum phase time is spent blocked on qubit connectivity rather than computing, and turnaround per job becomes heavily tailed.**

***Index Terms*—Quantum cloud simulations, quantum job scheduling, Hybrid-cloud simulation frameworks**

## I. Introduction

In recent years, quantum computing has transitioned from a theoretical concept [1] to a rapidly maturing technology that garners academic research [2] and industrial interest. According to the MIT Quantum Index Report 2025 [3], the field is experiencing an unprecedented expansion in hardware availability, corporate communications, and global intellectual property, evidenced by a fivefold increase in quantum technology patents over the past decade. The rapid development of commercial Quantum Processing Units (QPUs) highlights the urgent need for hybrid infrastructure capable of orchestrating classical and quantum workloads.

However, raw quantum advantage cannot be realized in isolation. Modern quantum algorithms, such as the Variational Quantum Eigensolver (VQE) [4], Quantum Approximate Optimization Algorithm (QAOA), and Quantum Neural Networks (QNNs), are inherently hybrid. These workloads depend on tight feedback loops where classical High Performance Computing (HPC) nodes handle statevector simulation, parameterized circuit compilation, and numerical optimization, while the QPUs execute quantum state preparations and measurements. As a result, the ultimate system throughput is constrained not only by QPU fidelity or qubit counts, but also by the efficient orchestration, dynamic scheduling, and data transfer overheads between classical CPU resources and heterogeneous QPU coprocessors.

This work is motivated by the need to model and quantify energy and power consumption within hybrid quantum cloud architectures. Because physical hardware access is constrained and energy dynamics in hybrid environments are complex, simulation serves as a vital tool. It allows researchers and system architects to model complex execution workflows, test scheduling hypotheses, and predict performance and energy metrics across scale. To bridge this critical tooling gap, this paper leverages HybridCloudSim, a discrete event simulation framework engineered to model, evaluate, and optimize hybrid classical quantum cloud environments.

Our contributions in this paper are as follows:

**Unified resource scheduling and brokerage.** A broker that schedules concurrent, heterogeneous jobs under strict capacity constraints, allocating CPU units and memory bandwidth classically while resolving quantum allocation at two levels, namely aggregate qubit capacity and physical connectivity, so that fragmentation effects invisible to qubit count only models are exposed.

**Energy and power accounting.** A configurable cost model that attributes energy per execution phase from device specific power profiles drawn from Ezratty et al. [5] as the ground truth.

**Integrated telemetry and monitoring.** An event driven monitor that records device utilization and instantaneous fleet power at event boundaries, complemented by per phase job records of computation and occupancy, producing traces that support analysis over the run itself rather than end of run aggregates.

Through a series of simulation studies, we demonstrate how HybridCloudSim enables system architects to locate hardware bottlenecks, evaluate scheduling policies, and reason about the energy and cost implications of resource allocation in emerging hybrid cloud infrastructures. In particular, we show that energy and power aware simulation provides a principled basis on which researchers and cloud operators can set pricing policy, by varying device configurations and workload parameters within the framework rather than by procuring physical hardware. Detailed circuit execution is deliberately abstracted away, so that the framework to act as a lightweight digital twin of a hybrid quantum classical cloud or hybrid cloud. As this work targets orchestration level modeling, we refer the reader to Nielsen and Chuang [6] for a detailed introduction to quantum computation and quantum information theory.

SC26 Workshops, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-1221-5/26/$31.00 ©2026 IEEE

TABLE I
ENERGETIC REQUIREMENTS AND ARCHITECTURAL COMPARISON OF QUANTUM COMPUTING MODALITIES (SOURCE: EZRATTY ET AL. [5]).

| **Qubit Type** | **Supercond.** | **Silicon** | **NV centers** | **Majorana** | **Photons** |
|---|---|---|---|---|---|
| cryogeny | 16–105 kW | 12 kW | $<$ 1 kW | 16 kW | 3 kW |
| qubits gate controls | from 20 mW to 100W/ qubit depending on architectures with microwave generation outside or inside the cryostat | | N/A | $<$ 25 mW/qubit | 300W for photons sources and detectors, qubit gates controls |
| computing | 1 kW | 1 kW | $<$1 kW | 1 kW | 700 W |
| # qubits used | 53–433 | 12 | $<$10 | N/A | 20 |
| **total** | **25 – 140 kW** | **21 kW** | **N/A** | **N/A** | **4 kW** |

## II. RELATED WORK

Classical cloud simulators, such as CloudSim [7], iCanCloud [8], and SimGrid [9], are designed to evaluate resource management and scheduling without physical infrastructure [10]. Many of these foundational tools have been extended to support specific operational needs; for example, WorkflowSim [11] adds complex scientific workflow modeling, while frameworks like GreenCloud [12] and E-mc$^2$ [13] introduce formal energy modeling for data centers [14]. Collectively, they demonstrate the utility of discrete event simulation for analyzing system dynamics and computational accounting.

As quantum computing increasingly adopts a cloud-based delivery model dedicated simulation toolkits have arisen to model hybrid quantum environments. Frameworks such as iQuantum [15], QSimPy [16], [17], and QCloudSim [18] apply discrete event simulation to evaluate quantum task scheduling, backend selection, and resource management strategies.

While simulation frameworks are critical for exploring these emerging architectures, existing quantum cloud simulators [19]–[21] focus primarily on performance, resource allocation, and queueing. They generally lack explicit energy and cost estimation at the workflow level. Furthermore, many frameworks [22], [23] treat quantum jobs as monolithic, overlooking the interleaved QPU and CPU execution phases central to hybrid algorithms.

Prior work shows that practical quantum overhead is driven by workload traits, backend variability, repeated sampling, and classical-quantum interactions. Senapati et al. [24] showed hardware calibration drift degrades workload reproducibility across backends. Related research characterized noise-sensitive states [25], [26] and devised predictive NISQ backend selection for quantum machine learning [27]. Subsequent uncertainty quantification studies analyzed execution variability, stability, and backend selection for variational quantum algorithms and VQE [28], [29]. Additional work across hybrid molecular simulation [30], machine learning translation [31], and variational/signal processing workloads [32] confirms that total execution cost is determined by the end-to-end hybrid workflow—including iterative classical optimization, repeated executions, and backend choice—rather than isolated quantum circuits.

Energy consumption is a critical concern in high performance computing (HPC) [33], [34], driving interest in quantum computing as a potentially energy efficient complement [35], [36]. While classical supercomputers can consume hundreds of megawatt-hours daily, present day quantum processors operate in the kilowatt range. However, meaningful energy evaluation requires analyzing total system power integrated over execution time, as a quantum computer's footprint is dictated by its supporting infrastructure rather than gate level operations. As summarized in Table I [5], superconducting systems demand significantly more power due to their environmental control requirements than other quantum systems.

In superconducting platforms, energy consumption is overwhelmingly dominated by system level infrastructure. Total system power ranges from 25 kW to over 140 kW, depending on system configuration and qubit count [5]. The primary contributor is the cryogenic refrigeration required to maintain millikelvin temperatures, drawing between 16 kW and 105 kW. Additional components, such as classical computing overhead and qubit gate control electronics, contribute marginally (often around 1 kW) and are generally subsumed into the total system consumption.

Based on this observation, we developed an energy accounting model in which the energy consumed by a quantum job is proportional to the duration for which it computes on QPUs and CPUs. Under this abstraction, the energy consumption of a superconducting quantum job is approximated as $E_{\text{job}} \approx P_{\text{baseline}} \cdot t_{\text{compute}}$, where $P_{\text{baseline}}$ denotes the average system-level power consumption, including cryogenics and supporting electronics, and $t_{\text{compute}}$ represents the job's circuit execution time on the quantum device, exclusive of time spent waiting for qubits (Section IV). This baseline power model aligns with energy accounting practices commonly used in HPC systems, where always-on infrastructure costs are amortized over job execution time.

## III. SYSTEM ARCHITECTURE

HybridCloudSim is a discrete event simulator built on SimPy. It models a hybrid cloud at the *orchestration* layer: job arrival, device selection, capacity blocking, iterative quantum classical execution, and the resulting energy and cost. Circuit level quantum behavior is deliberately abstracted; device service times are obtained from analytical performance models parameterized by published hardware specifications, so that a full run over thousands of jobs completes in seconds to minutes on a single core.

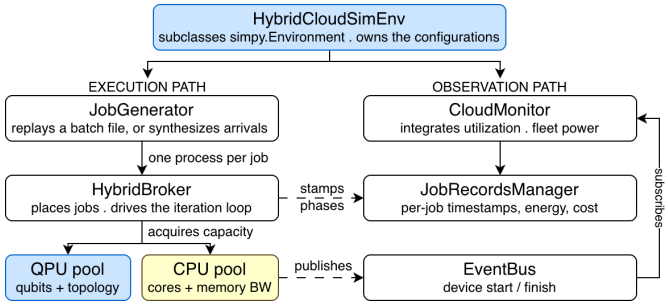



Fig. 1. Component structure of HybridCloudSim. Both paths descend from the composition root, but they never join: the execution path (left) places jobs and consumes device capacity, while the observation path (right) only records what the execution path has already committed. Dashed links carry telemetry and never influence placement.

TABLE II
PRINCIPAL COMPONENTS OF HYBRIDCLOUDSIM.

| Component | Responsibility |
|---|---|
| `HybridCloudSimEnv` | Composition root; owns the cost model |
| `JobGenerator` | Trace replay or stochastic arrivals |
| `HybridBroker` | Per job scheduling and phase control |
| `QuantumDevice` | Qubit capacity, topology, QPU time |
| `CPU` | CPU and memory bandwidth capacity |
| `EventBus` | Decoupled device event notification |
| `CloudMonitor` | Utilization integration, fleet power |
| `JobRecordsManager` | Event log, and job metrics |

### A. Overview

The simulation framework is organized around a single composition root, `HybridCloudSimEnv`, which subclasses the SimPy environment and instantiates every other component. Table II summarizes the roles. Control flows top down, where the job feed creates one broker process per job, and each broker drives devices. Observability flows bottom up: devices emit events on a publish subscribe bus and write per job records that the monitor and the records manager aggregate.

A run is fully specified by three inputs: a device inventory (a list of QPU and CPU objects), a workload (a trace file or an arrival process), and a cost configuration. Devices are constructed detached from the environment and are bound to it in a separate initialization pass, at which point their SimPy containers and resources are created. This two phase construction keeps device definitions declarative and makes parameter sweeps explicit: because devices carry mutable state, specifically qubit containers, the topology graph, and the qubit occupancy map, a fresh inventory is instantiated for each configuration in a sweep.

Figure 1 makes this separation explicit. The execution and observation paths descend from the same root but never join: the broker consults device capacity in order to place work, whereas the event bus, the cloud monitor, and the records manager only ever read state that the execution path has already committed. No scheduling decision depends on a telemetry value. This asymmetry is what allows instrumentation to be as dense as the analysis requires; every phase transition of every job is stamped, and every device transition is published, without perturbing the schedule under study. It also fixes the provenance of each reported quantity: phase boundaries originate in the broker, device arrivals in the devices themselves, time integrated utilization and instantaneous fleet power in the monitor, and per job energy and cost in the records manager. We rely on that provenance in Section IV-D, where the two independently computed energy figures are compared.

**Algorithm 1** Broker process for job $J$

$i \leftarrow 0$
**while** $i < r$ **do**
  $q \leftarrow \textsc{Select}(\text{QPU}, n)$
  **while** $q = \text{null}$ **do** ▷ no device has free capacity
    **wait** $\delta$; $q \leftarrow \textsc{Select}(\text{QPU}, n)$
  **end while**
  $\textsc{Execute}(q, J)$; $\textsc{Record}(J, \text{QPU})$
  $u \leftarrow \textsc{Select}(\text{CPU}, (c, b))$
  **while** $u = \text{null}$ **do**
    **wait** $\delta$; $u \leftarrow \textsc{Select}(\text{CPU}, (c, b))$
  **end while**
  $\textsc{Execute}(u, J)$; $\textsc{Record}(J, \text{CPU})$
  $i \leftarrow i + 1$
**end while**
$\textsc{FinalizeEnergyAndCost}(J)$

### B. Workload

A job is a tuple $J = (n, d, s, p, a, r, c, b)$ giving qubit count $n$, circuit depth $d$, shot count $s$, priority $p$, arrival time $a$, required iterations $r$, CPU units $c$, and memory bandwidth $b$. The shipped broker does not consult $p$, and every trace in this paper sets it to a constant. Jobs enter the system in one of two modes. In *dispatcher* mode, the generator replays a CSV or JSON trace, releasing each job at its recorded arrival time; this is the mode used for all experiments in this paper, since it makes workloads reproducible and shareable. In *generator* mode, jobs are synthesized online from a configurable inter arrival distribution (exponential by default) with randomized circuit parameters. In both modes the generator spawns one broker process per job, so concurrency in the model is bounded by device capacity rather than by any fixed pool of workers.

### C. Broker and Execution Lifecycle

The broker is the scheduling core. Each job executes $r$ variational iterations, and every iteration consists of a QPU phase followed by a CPU phase. The structure of VQE and QAOA style workloads in which measurement outcomes are returned to a classical optimizer that produces the next parameter set. Algorithm 1 states the loop.

SELECT filters the inventory to devices of the requested type that are not under maintenance and whose free capacity meets the request, namely free qubits for a QPU, free units *and* free memory bandwidth for a CPU, and returns the candidate with the most free capacity, for a CPU the most free units and then the most free bandwidth. When no candidate exists,

the broker retries after a fixed polling interval $\delta$ rather than enqueueing, which keeps the admission decision re-evaluated against current fleet state. The resulting blocking behavior, rather than an explicit queue discipline, is what produces the contention observed in Section VI. The policy is localized in SELECT, so alternative disciplines (priority, fair share, energy aware placement) can be substituted without touching the phase loop.

## IV. MODELS

We adopt an energy accounting model inspired by practices in HPC [37]. Electrical energy consumption is defined as the time integral of instantaneous power consumption,

$$E = \int_{t_s}^{t_e} P(t)\, dt,$$

where $P(t)$ denotes the system power draw and $[t_s, t_e]$ represents the execution interval of a job.

For superconducting quantum computers, system power is dominated by the cryogenic refrigeration and supporting infrastructure required to operate continuously and maintain qubit coherence [38]. However, while this baseline cooling requirement is massive, the total power consumption is not strictly independent of workload characteristics [39]. We approximated instantaneous power consumption as constant during operation,

$$P(t) \approx P_{\text{baseline}}.$$

Under this assumption, the energy consumption of a quantum job simplifies to

$$E_{\text{job}} \approx P_{\text{baseline}} \cdot (t_e - t_s),$$

where $P_{\text{baseline}}$ assumes the average power consumption, including cryogenics, control electronics, and classical infrastructure.

This accounting model simplifies energy attribution without relying on fine-grained gates or pulses, which are difficult to obtain and validate for current quantum hardware.

The parameters used to model energy consumption in this experiment are summarized in Table III. Given these parameters, the energy consumption of a quantum job is computed as

$$E_{\text{job}} = P_{\text{baseline}} \cdot t_{\text{compute}},$$

where each job is charged the full baseline for its own computation regardless of how many tenants the device hosts concurrently; per job energy is therefore a sole tenant attribution.

Each job consists of a sequence of execution stages that alternate between QPU and CPU devices. For a given job $j$, execution on QPUs and CPUs is recorded as ordered lists of start and finish timestamps:

$$\{\texttt{qpu_start}_i^j, \texttt{qpu_finish}_i^j\}_{i=0}^{N_q-1}$$

$$\{\texttt{cpu_start}_i^j, \texttt{cpu_finish}_i^j\}_{i=0}^{N_c-1}$$

TABLE III
ENERGY RELATED PARAMETERS FOR SUPERCONDUCTING QUANTUM DEVICES

| Parameter | Unit | Description |
|---|---|---|
| $P_{\text{baseline}}$ | kW | Average power consumption, including cryogenic refrigeration, control electronics, and classical infrastructure. |
| $t_{\text{compute}}$ | h | Circuit execution time of a quantum job on the quantum processing unit, excluding time spent waiting for a connected qubit region. |
| $E_{\text{job}}$ | kWh | Energy consumption attributed to an individual quantum job. |

where $N_q$ and $N_c$ denote the number of QPU and CPU execution segments, respectively.

Each QPU segment corresponds to a quantum execution phase, while each CPU segment represents classical preprocessing, post-processing, or intermediate computation required by the hybrid algorithm.

The window $\texttt{qpu_finish}_i^j - \texttt{qpu_start}_i^j$ of the $i$-th QPU segment also contains the time the job spends waiting for a connected qubit region (Section IV-A); it is retained as the segment's occupancy, while the execution time billed for energy is the circuit execution time recorded by the device,

$$t_i^{\text{qpu}} = \texttt{qpu_compute}_i^j,$$

and the execution time of the $i$-th CPU segment is

$$t_i^{\text{cpu}} = \texttt{cpu_finish}_i^j - \texttt{cpu_start}_i^j.$$

In that way, quantum and classical execution phases are interleaved multiple times within a single job.

### A. Two-Level QPU Allocation

QPUs are the sole spatially constrained resource in the model. Each device loads a topology graph $G = (V, E)$ ($|V|$ physical qubits) and maintains a qubit occupancy map. Admission requires a two-level check: (1) an aggregate counter showing at least $n$ free qubits, and (2) a breadth-first search that grows a *connected* region of $n$ free qubits from the first feasible start node. Upon allocation, selected qubits are marked busy and their adjacent edges cut; upon completion, qubits are freed and cut edges restored between available pairs.

Unlike capacity-only abstractions, this two-level model captures qubit fragmentation: a job meeting the aggregate threshold can still fail admission if free qubits are disconnected, reflecting multi-tenant behavior on sparsely connected superconducting lattices.

QPU service time follows a throughput model based on the vendor-reported Circuit Layer Operations Per Second (CLOPS) metric:

$$t_{\text{QPU}} \;=\; \alpha\, \frac{MKS \log_2 d}{\text{CLOPS}}, \tag{1}$$

where $S$ is the shot count, $d$ the circuit depth, $M$ and $K$ the number of circuit templates and parameter updates per submission, and $\alpha$ a unit conversion constant; the devices used

here set $M = K = 1$ and $\alpha = 10$. Calibration tables (readout, single-qubit, and two-qubit gate errors) are additionally loaded and a circuit fidelity estimator is provided, but fidelity is neither reported nor fed back into scheduling in the present study.

### B. Classical Device Model

Classical nodes expose two independent capacities, compute units and memory bandwidth, both of which must be satisfied for a job to be admitted; modeling bandwidth separately is necessary because statevector post-processing is frequently bandwidth- rather than compute-bound. The default node derives its service time from the algorithmic cost of the classical half of the variational loop:

$$t_{\mathrm{CPU}} \;=\; \frac{\kappa\, 2^{\,n} d\, \pi}{\theta\, c^{\,0.85}}, \qquad \theta = \tfrac{1}{2}\big(f_{\mathrm{base}} + f_{\mathrm{boost}}\big) \cdot \mathrm{IPC}, \quad (2)$$

where $\pi$ is the variational parameter count ($\pi = 1$ in every experiment reported here, since trace replay carries no parameter count), $\kappa$ a calibration constant, $\theta$ the effective per-core throughput derived from clock and IPC, and the exponent $0.85$ captures sublinear scaling with allocated units. The $2^{\,n}$ term makes the classical side of the loop grow exponentially with qubit width, which is the mechanism by which classical post-processing overtakes quantum execution as circuits widen. A simpler node with randomized service time is also provided for baseline comparisons.

### C. Energy and Cost Model

Power parameters are set via a single environment configuration with per-device overrides. QPUs draw constant power $P^q$ (dominated by load-independent cryogenic cooling) whenever holding qubits for a job, while classical nodes follow an affine power model between an idle floor and a peak, driven by utilization $u_j$. Instantaneous fleet power is:

$$P(t) \;=\; \sum_{q \in \mathcal{Q}} P^q \mathbb{1}[q \text{ busy at } t] + \sum_{j \in \mathcal{C}} \Big[ P^j_{\mathrm{idle}} + \big(P^j_{\mathrm{peak}} - P^j_{\mathrm{idle}}\big) u_j(t) \Big]. \quad (3)$$

Per-job energy is attributed by phase. For job $J$ with QPU computation times $t^q_i$ and CPU phase durations $t^c_i$ on the devices selected in iteration $i$,

$$E(J) \;=\; \sum_{i=1}^{r} \Big( P^q_i t^q_i + P^c_i t^c_i \Big), \qquad \mathrm{Cost}(J) \;=\; p_e\, E(J), \quad (4)$$

with $p_e$ the electricity price per kWh and $P^c_i$ the node's peak draw. Equations (3) and (4) are complementary rather than redundant: the former gives the operator's instantaneous load, which determines provisioning and peak demand charges, while the latter gives the per-tenant attribution needed to set prices. Both are driven by the same per device profile, the rate at which a phase is billed being also the peak of Eq. (3), so a change to a device's power profile propagates to both views.

### D. Telemetry

Devices stream `device_start` and `device_finish` events. On each event, the cloud monitor integrates resource-time metrics to record exact utilization and power (3). Exact event-boundary sampling scales with system activity rather than wall-clock time. Concurrently, a records manager logs timestamps to track latency and compute final makespan, energy, and cost (4). Storing per-iteration traces supports both cross-job and within-job analysis. Aggregated job energy is compared with the integrated power series, which measures fleet occupancy rather than per tenant computation and is not expected to match it.

## V. System Metrics: Instantaneous vs. Cumulative Utilization

To evaluate the operational efficiency and temporal dynamics of the hybrid scheduler, we distinguish between *instantaneous resource load* and *cumulative resource utilization*. While instantaneous load measures real-time demand variations and queue drainage, cumulative utilization captures the time-integrated efficiency of the heterogeneous hardware over the entire operational lifespan of the simulation.

### A. System Capacity Definitions

Let $\mathcal{D}_{\mathrm{QPU}}$ and $\mathcal{D}_{\mathrm{CPU}}$ represent the sets of quantum processing units (QPUs) and classical central processing units (CPUs), respectively. The static global capacity of each resource type is defined as:

$$C_{\mathrm{QPU}} = \sum_{d \in \mathcal{D}_{\mathrm{QPU}}} c(d), \quad C_{\mathrm{CPU}} = \sum_{d \in \mathcal{D}_{\mathrm{CPU}}} c(d), \quad C_{\mathrm{MEM}} = \sum_{d \in \mathcal{D}_{\mathrm{CPU}}} b(d) \quad (5)$$

where $c(d)$ denotes the resource unit capacity of device $d$ (qubits for a QPU, CPU units for a classical node), and $b(d)$ represents the associated memory bandwidth capacity.

### B. Instantaneous Resource Load

At any given simulation time $t \in [0, T]$, the instantaneous load $L_k(t)$ for a resource type $k \in \{\mathrm{QPU}, \mathrm{CPU}, \mathrm{MEM}\}$ measures the fraction of hardware capacity actively committed to processing workloads. Let $u_k(t)$ be the total units of resource $k$ allocated at time $t$. The instantaneous load percentage is defined as:

$$L_k(t) = \left( \frac{u_k(t)}{C_k} \right) \times 100\% \quad (6)$$

When the job queue depletes and all active workloads complete at time $t_{\mathrm{idle}} \leq T$, the active allocation drops to zero ($u_k(t) = 0$), yielding:

$$\lim_{t \to t^+_{\mathrm{idle}}} L_k(t) = 0\% \quad (7)$$

Thus, $L_k(t)$ is non-monotonic and exhibits sharp step-wise transitions corresponding to job arrivals and completions.

### C. Cumulative Resource Utilization

Whereas instantaneous load evaluates active demand at a discrete millisecond, cumulative utilization $U_k(T)$ quantifies total resource consumption relative to total available resource-time product over the entire elapsed simulation window $[0, T]$.

The cumulative workload metric is formulated as the integral of active unit allocations over time:

$$W_k(T) = \int_0^T u_k(t)\, dt \tag{8}$$

The total available resource capacity integrated across time $T$ is given by:

$$\Omega_k(T) = C_k \cdot T \tag{9}$$

The cumulative percentage utilization $U_k(T)$ up to time $T$ is therefore expressed as:

$$U_k(T) = \left(\frac{W_k(T)}{\Omega_k(T)}\right) \times 100\% = \left(\frac{\int_0^T u_k(t)\, dt}{C_k \cdot T}\right) \times 100\% \tag{10}$$

### D. Discrete Event Formulation

In a discrete-event simulation paradigm (such as SimPy), state transitions occur at discrete timestamps $t_0, t_1, t_2, \ldots, t_N$. The continuous integral $W_k(T)$ is evaluated exactly via zero-order hold accumulation across intervals $\Delta t_i = t_i - t_{i-1}$:

$$W_k(T) = \sum_{i=1}^{N} u_k(t_{i-1}) \cdot (t_i - t_{i-1}) \tag{11}$$

Unlike instantaneous load $L_k(t)$, the cumulative utilization $U_k(T)$ does not drop abruptly to zero upon workload completion. Instead, if the system remains idle beyond $t_{\text{idle}}$, $W_k(T)$ remains constant while the denominator $\Omega_k(T)$ grows linearly with $T$, causing $U_k(T)$ to exhibit a smooth, asymptotic decay toward zero:

$$U_k(T) = \left(\frac{W_k(t_{\text{idle}})}{C_k \cdot T}\right) \times 100\%, \quad \forall\, T \geq t_{\text{idle}} \tag{12}$$

## VI. Use Cases

We design a representative use case that reflects workload heterogeneity, resource contention, and energy cost interactions across quantum and classical subsystems.

We generate a synthetic workload consisting of 1,200 independent quantum circuit synthesis jobs, stored in a CSV file and submitted to the simulator as an external job trace.

Table IV summarizes the characteristics of the workload, infrastructure, and cost configuration used in our hybrid quantum classical cloud simulation. This consolidated view is intended to improve reproducibility and provide readers with a concise overview of the experimental setup. Here, *cpu units* and *memory units* represent normalized per thread compute capacity and per GB/s bandwidth capacity, respectively. The trace's classical columns are recorded but not consulted by the dispatcher: as in Section VII-B, every CPU phase is admitted against a fixed budget of 8 CPU and 20 memory bandwidth units.

TABLE IV
HybridCloudSim Use Case Configurations and Workload Parameters

| Category | Configuration |
|---|---|
| Workload size | 1,200 hybrid synthesis jobs (CSV trace) |
| Quantum workload parameters | Number of qubits: 10 to 25 |
| | Circuit depth: 37 to 99 |
| | Shots per job: 1,000 to 5,000 |
| | Iterations per job: 4 to 9 |
| | Arrival process: Exponential ($\lambda \approx 0.36$) |
| Classical workload parameters | CPU units: 4 to 7 (qubit scaled) |
| | Memory BW units: 4 to 7 (qubit scaled) |
| Quantum resources | 2 IBM Eagle r3 QPUs (127 qubits each) |
| | IBM_Quebec: 32,000 CLOPS |
| | IBM_Kyiv: 30,000 CLOPS |
| Classical resources | 1 CPU nodes |
| | 8 cores, 16 threads per node |
| | Base / boost: 3.8 / 5.0 GHz |
| | Memory bandwidth: 51.2 GB/s |
| Resource abstraction | CPU capacity = threads $\times$ cpu units |
| | Mem BW capacity = GB/s $\times$ mem units |
| Energy pricing | \$0.18 per kWh |
| QPU power model | Static power draw: 50 kW each |
| CPU power model | Per job billing: constant at peak |
| | CPU peak: 5.0 kW |
| | Fleet power: affine, idle 30% of peak |

We assume a fixed electricity price of \$0.18 per kWh, representative of commercial cloud pricing. Quantum devices are modeled with static power draw values to reflect the high baseline energy cost of superconducting systems, with each QPU assigned a power draw of 50 kW. These values are selected within the reported power consumption range of superconducting quantum computing systems [5].

A classical CPU node is assigned a device specific power envelope of 5.0 kW, at which each job is billed for the duration of its own computation, while the node's instantaneous draw in the fleet view follows an affine model between an idle floor of 30% of that envelope and the envelope itself. We note that absolute CPU power values are less critical in our study, as our results are driven by the relative energy imbalance between QPU and classical resources. The detailed models are provided in Section IV.

### A. Performance Evaluation

In this section, we evaluate the performance characteristics of the HybridCloudSim environment setup and analyze aggregate cloud resource usage to understand overall system behavior, followed by a time series analysis to capture temporal utilization dynamics.

In terms of cost, the mean energy cost per job is \$0.09956 $\pm$ 0.04935, with a median cost of \$0.0939 and a 95th percentile cost of \$0.19052, reflecting the right skewed cost distribution. Energy fraction statistics further confirm the dominant role of quantum execution, with a mean QPU energy fraction of 0.98194 $\pm$ 0.07942 and a mean CPU energy fraction of 0.01805 $\pm$ 0.07942. These results quantitatively reinforce the earlier observation that overall energy consumption and cost in hybrid workloads are overwhelmingly driven by QPU operation, while classical resources contribute smaller but non negligible overheads.

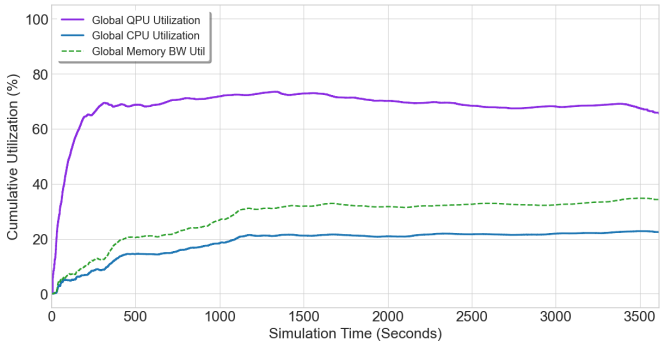



Fig. 2. Cumulative Cloud Resource Utilization Over Time.

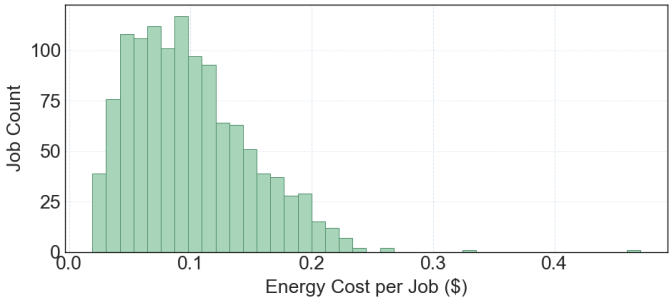



Fig. 3. Distribution of energy cost per job ($) across executed workloads, showing a right-skewed distribution where the majority of jobs incur costs under $0.20, with a small tail extending up to $0.48.

### B. Time Series Resource Utilization

Figure 2 illustrates the cumulative time integrated resource utilization across the hybrid quantum cloud environment over a 3,764 second execution window. During the initial warm up phase ($t < 500\,\text{s}$), all three subsystem resources experience a steep ramp up in utilization as arriving workloads fill the system pipelines. The global quantum processing unit (QPU) utilization rises rapidly, stabilizing into a steady state regime between $67\%$ and $73\%$ between $t = 500\,\text{s}$ and $t = 3,500\,\text{s}$. This sustained high demand highlights the QPU as the primary performance bottleneck within the hybrid infrastructure. In contrast, classical resource demands stabilize at lower levels, with memory bandwidth cumulative utilization settling between $30\%$ and $35\%$ after $t \approx 1{,}200\,\text{s}$ and global CPU utilization settling near $20\%$ to $23\%$. The pronounced gap between QPU and CPU or memory utilization reflects the asymmetric nature of hybrid workflows, where classical hardware spends significant time idle while waiting for long duration quantum circuit executions to complete. The detailed calculations are provided in Section V.

### C. Energy Consumption and Cost Analysis

We next analyze the energy characteristics of the hybrid cloud environment.

*1) Distribution of Energy Cost Across Jobs:* Figure 3 shows the distribution of energy cost per job, computed using the modeled energy consumption and a fixed electricity price. The

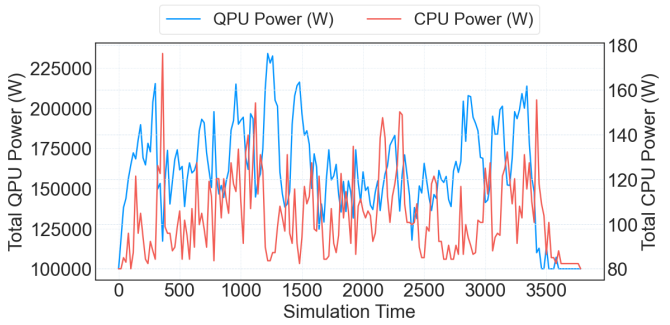



Fig. 4. Time-series profile of QPU (left y-axis, blue) and CPU (right y-axis, red) power consumption in Watts over total simulation time (seconds) under the load dependent power variant described in the text, illustrating the dynamic resource utilization and power fluctuations during hybrid quantum-classical execution.

distribution is right skewed, with most jobs clustered at lower costs and a diminishing number of higher cost outliers.

This behavior arises from variability in circuit depth, qubit count, execution duration, and iterative workload structure. Importantly, the absence of multimodal behavior suggests that cost variation is driven by continuous workload heterogeneity rather than discrete execution regimes.

*2) Time Series Power Consumption:* Figure 4 illustrates the time series power consumption of QPU and CPU resources. It is drawn from the job records by an illustrative load dependent variant of the power model rather than by Eq. (3): the two QPUs draw their combined 100 kW baseline rising linearly toward 300 kW with the fraction of a fixed 640 qubit reference capacity claimed by admitted jobs, each counted from admission to release, and the CPU node draws 80 W rising toward 360 W with the 1.3 power of its unit utilization. Because admitted jobs are counted before a connected region is secured, the claimed fraction measures demand pressure on the fleet rather than physical qubit occupancy, and under contention it exceeds what 254 qubits could hold. The QPU power remains persistently high, fluctuating within a range of approximately 100 to 235 kW around its baseline operating level. These variations coincide with active execution periods, but they sit on top of a fixed 100 kW cryogenic cooling and control baseline that the two QPUs draw even when idle, which alone accounts for over 60% of the 160 kW mean draw.

In contrast, CPU power operates at a significantly lower scale on this variant's 80 to 360 W envelope, distinct from the 5.0 kW billing rate, varying between roughly 80 and 175 W over time. Unlike the QPU, CPU power exhibits more pronounced relative fluctuations that correlate with classical execution phases, job arrivals, and orchestration activities. This stark separation in power magnitude between QPU and CPU resources indicates that energy optimization in hybrid quantum classical environments is primarily driven by QPU utilization efficiency rather than classical compute scaling.

The plotted series spans an aggregate QPU power draw of approximately 100 to 235 kW for a configuration consisting of two superconducting QPUs, corresponding to roughly 50 to

TABLE V
SIMULATION CONFIGURATION AND WORKLOAD CHARACTERISTICS FOR HYBRID WORKFLOW BEHAVIOR.

| Category | Configuration |
|---|---|
| Workload size | 3,000 synthesis jobs |
| Quantum workload parameters | Number of qubits: 10 to 20 |
| | Circuit depth: 10 to 20 |
| | Shots per job: 2,000 to 9,000 |
| | Arrival process: Exponential ($\lambda \approx 1.04$) |
| Classical workload parameters | CPU units: 7 to 10 (qubit scaled) |
| | Memory BW units: 14 to 20 (qubit scaled) |
| Quantum resources | 2 QPUs (127 qubits each) |
| Classical resources | 2 CPU nodes |
| | 8 cores, 16 threads per node |
| | Base / boost: 3.8 / 5.0 GHz |
| | Memory bandwidth: 51.2 GB/s |
| Resource abstraction | CPU capacity = threads $\times$ cpu units |
| | Mem BW capacity = GB/s $\times$ mem units |
| Energy pricing | \$0.18 per kWh |
| QPU power model | Static power draw |
| | QPU 1: 70 kW |
| | QPU 2: 60 kW |
| CPU power model | Per job billing: constant at peak |
| | CPU 1 peak: 5.0 kW |
| | CPU 2 peak: 6.5 kW |
| | Fleet power: affine, idle 30% of peak |

117 kW per QPU. We note that these results are contingent on the chosen configuration parameters, taking Ezratty et al. [5] as the baseline reference in Table I. Modifying the system or workload inputs will naturally alter the simulation outcomes. This sensitivity is a key advantage of our highly configurable framework.

## VII. HYBRID WORKFLOW BEHAVIOR

Many near term quantum applications are realized not as single shot quantum programs, but as *iterative hybrid quantum classical workflows*. Representative examples include the Variational Quantum Eigensolver (VQE), the Quantum Approximate Optimization Algorithm (QAOA), and hybrid calibration and tuning routines deployed in contemporary quantum cloud environments.

To isolate the impact of hybrid workflow structure on system behavior, we conduct a controlled experiment in which physical resources, circuit characteristics, and workload composition are held constant, while only the number of quantum classical iterations per job is varied.

The experimental setup is listed in Table V. We use a fixed set of 3,000 synthesized quantum jobs generated with identical distributions for qubit counts, circuit depth, shot count, and classical resource requirements. The hybrid environment consists of two QPUs and two CPU devices, operated under the same scheduling policy and cost model parameters across all experimental runs.

The sole parameter that differs between experiments is the required number of hybrid iterations per job. Specifically, we evaluate seven workflow configurations with iteration counts ranging from 3 to 21 in increments of 3. Each configuration is provided as a separate workload trace. The purpose is to assure that any observed differences in execution time, energy consumption, or cost are attributable exclusively to changes in workflow structure rather than hardware variability or workload diversity.

### A. Iteration Scaling and the Onset of Quantum Congestion

Variational algorithms (e.g., VQE, QAOA) iteratively alternate between $k$ circuit execution rounds and classical parameter updates. Standard hybrid cloud cost models assume a linear multiplier where $k$ iterations cost $k\times$ a single iteration; the assumption holds only for the computation itself. Beyond a critical iteration count, the time a job occupies a QPU per iteration escalates by more than an order of magnitude while its computation, and the classical phase, scale perfectly linearly. This effect stems from contention across a finite quantum fleet rather than from the application; it is invisible to models that ignore device occupancy and misattributed by those that bill the occupied window as computation.

### B. Experimental Design

The experiment is a single factor sweep. We generate one batch of $N = 3000$ synthetic variational jobs and replicate it seven times, altering exactly one field, namely the requested iteration count, the $r$ of Section III-B, written $k \in \{3, 6, 9, 12, 15, 18, 21\}$ here. Every other job attribute is bit identical across the seven traces: qubit width, circuit depth, shot count, priority, arrival timestamp, CPU unit demand, and memory bandwidth demand. Any difference in the measured outcome is therefore attributable to $k$ alone, with no confounding from workload composition or arrival pattern.

Jobs request 10 to 20 qubits (mean 15.15), circuit depth 10 to 20, 2000 to 9000 shots (mean 5501), 7 to 10 CPU units, and 14 to 20 memory bandwidth units. Arrivals are replayed over a window of 2884.8 simulation seconds, a mean rate $\lambda \approx 1.04$ jobs per second. The classical columns are not consulted: every CPU phase is admitted against a fixed budget of 8 CPU and 20 memory bandwidth units and runs on a CPU unit allocation drawn uniformly from 4 to 16, the sweep's sole stochastic element.

The target platform is a fixed hybrid inventory: two superconducting QPUs modeled on IBM Eagle r3 devices (`IBM_Strasbourg` as QPU 1 and `IBM_Brussels` as QPU 2, 127 physical qubits each on the heavy hex coupling map, with their published CLOPS; quantum volume and $T_1/T_2$ are recorded but unused), and two `AMDRyzen` classical nodes, instantiated afresh for every run so that no qubit occupancy or topology state leaks between configurations. Energy is attributed per phase from constant device draws of 70 kW (QPU 1), 60 kW (QPU 2), 5.0 kW (CPU 1), and 6.5 kW (CPU 2) at \$0.18 per kW h. Each configuration runs to completion and is summarized over all 3000 jobs.

Two structural features of the platform matter for what follows. First, a QPU admits several jobs concurrently, bounded not by a job slot count but by free qubits: at a mean width of 15.15 qubits, the 254 qubit fleet could in principle host $C \approx 16.8$ jobs at once. Second, admission is resolved at two levels, specifically aggregate qubit capacity *and* physical connectivity. A job is admitted only when a *connected* subgraph

of free qubits exists, so the achievable concurrency is strictly below $C$ whenever the free qubits are scattered across the coupling graph.

It is convenient to summarize the concurrency demand by the dimensionless ratio

$$\rho(k) \;=\; \frac{\lambda\, s_1\, k}{C}, \tag{13}$$

where $s_1 = 0.971\,\mathrm{s}$ is the QPU computation time of a single iteration, identical at every sweep point. Equation (13) is the fraction of nominal fleet concurrency that the workload demands; it rises from $\rho = 0.18$ at $k = 3$ to $\rho = 1.26$ at $k = 21$ and passes unity at $k \approx 16.6$.

### C. Metrics and Figure Construction

For each QPU phase the device records its *computation* time $T_{\mathrm{comp}}$, the circuit execution time of Eq. (1), while the broker stamps its *occupancy* $T_{\mathrm{occ}}$, the window from admission at the capacity gate to release, throughout which the job is resident on a powered device; they differ by the time spent retrying the connectivity check of Section IV-A, once per second, until a connected free region exists. Per job energy is charged on $T_{\mathrm{comp}}$ alone; it is the QPU phase duration entering Eq. (4). Since Eq. (1) depends only on job attributes, $\sum T_{\mathrm{comp}}$ is linear in $k$ *by construction*: a control, not a finding. Figure 5 therefore reports three *derived*, scale free views of the sweep, each chosen so that the linear scaling null hypothesis appears as a flat line and departure from linearity as departure from flatness.

*a) Panel (a): computation and occupancy per iteration.:* We divide the mean per job QPU computation, QPU occupancy, and CPU phase time by $k$, the average cost of one iteration on each resource class. Under the multiplier assumption all three series are horizontal. All three share one logarithmic axis (s per iteration) because occupancy spans a factor of 40 while the other two do not move.

*b) Panel (b): compute fraction of occupied QPU time.:* For each run we form the ratio of sums

$$\eta \;=\; \frac{\sum T_{\mathrm{comp}}}{\sum T_{\mathrm{occ}}} \;\in\; (0, 1], \tag{14}$$

the fraction of reserved, powered QPU time that performs computation. $\eta = 1$ is exactly the multiplier assumption and serves as the reference; a second line at $\eta = \frac{1}{2}$ marks the point past which most occupied QPU time is idle.

*c) Panel (c): dispersion and tail weight.:* We plot two dimensionless descriptors of the per job turnaround distribution *within* each run: the coefficient of variation $\mathrm{CV} = \sigma/\mu$, and the tail ratio $p_{95}/\mathrm{median}$. Both are invariant to the overall time scale, so they isolate changes in the *shape* of the distribution from changes in its magnitude. The reference line at $\mathrm{CV} = 1$ marks exponential like dispersion, the canonical signature of a queue that has begun to build.

In all three panels a shaded band marks the congested regime, its left edge where $\eta$ falls through one half, interpolated inside the $12 \to 15$ bracket ($k \approx 13$). The figure is generated from the sweep summary by `plot_iteration_knee.py` without smoothing or fitting.

TABLE VI
ITERATION SWEEP OVER AN IDENTICAL 3000 JOB TRACE. $\rho$ IS THE CONCURRENCY DEMAND OF EQ. (13), PER ITERATION COLUMNS ARE MEAN PHASE TIME DIVIDED BY $k$, $\eta$ IS THE COMPUTE FRACTION OF EQ. (14), AND $E$ IS MEAN PER JOB ATTRIBUTABLE ENERGY. ROWS BELOW THE RULE LIE IN THE SHADED BAND.

| | | Per iteration (s) | | | | | Turnaround |
|---|---|---|---|---|---|---|---|
| $k$ | $\rho$ | $T_{\mathrm{occ}}$ | CPU | $\eta$ | $E$ (kW h) | CV | $p_{95}/\tilde{x}$ |
| 3 | 0.18 | 0.971 | 0.155 | 1.000 | 0.053 | 0.43 | 1.72 |
| 6 | 0.36 | 0.974 | 0.155 | 0.997 | 0.107 | 0.43 | 1.72 |
| 9 | 0.54 | 1.018 | 0.156 | 0.954 | 0.160 | 0.46 | 1.83 |
| 12 | 0.72 | 1.392 | 0.156 | 0.697 | 0.213 | 0.69 | 2.79 |
| 15 | 0.90 | 7.809 | 0.155 | 0.124 | 0.267 | 1.05 | 5.98 |
| 18 | 1.08 | 26.02 | 0.156 | 0.037 | 0.320 | 1.08 | 5.78 |
| 21 | 1.26 | 39.25 | 0.156 | 0.025 | 0.373 | 0.98 | 4.07 |

### D. Results

Computation is a control, and it stays flat. The most useful columns of Table VI are the two that do not move. QPU computation per iteration is $0.971\,\mathrm{s}$ at every $k$, and the measured classical phase is $0.155\,\mathrm{s}$ per iteration at $k = 3$ and $0.156\,\mathrm{s}$ at $k = 21$, constant to within $1\%$ across a sevenfold increase in loop length. Per job attributable energy follows, growing from $0.053$ to $0.373\,\mathrm{kW\,h}$, a factor of $6.99$ for $7\times$ the iterations, with a QPU share flat at $98.4\%$. This is correct linear scaling measured on the same runs, which forecloses a harness artifact as the explanation for what follows.

Through $k = 9$ occupancy also behaves as the multiplier assumption predicts: a job holds the QPU for $0.971$, $0.974$, and $1.018\,\mathrm{s}$ per iteration. Then it breaks. Occupancy per iteration rises to $1.39$ at $k = 12$, $7.81$ at $k = 15$, $26.0$ at $k = 18$, and $39.2\,\mathrm{s}$ at $k = 21$: a $40\times$ increase in the time a job is resident on a powered QPU per iteration against a $1.00\times$ change in the time it computes there. A 21 iteration job receives 21 iterations that each take far longer, because the fleet is congested by jobs like itself.

The compute fraction collapses near $k \approx 13$. $\eta$ is $1.000$ at $k = 3$, $0.954$ at $k = 9$, $0.697$ at $k = 12$, $0.124$ at $k = 15$, and $0.025$ at $k = 21$, falling through one half at $k \approx 13$. Past that knee more than half, and by $k = 18$ more than $96\%$, of the time a job spends resident on a cryogenically powered device is spent not computing: mean idle time per job is $5.1\,\mathrm{s}$ at $k = 12$, $103\,\mathrm{s}$ at $k = 15$, and $804\,\mathrm{s}$ at $k = 21$, against $20.4\,\mathrm{s}$ of QPU computation. The time to drain the batch rises from $2888\,\mathrm{s}$ to $4036\,\mathrm{s}$, and cumulative qubit utilization from $16\%$ to $89\%$.

Panel (c) confirms this from a quantity the first two panels never touch, the shape of the turnaround distribution. Through $k = 9$ turnaround is tight, with $\mathrm{CV} \le 0.46$ and a $p_{95}$ at most $1.83\times$ the median. The tail ratio reaches $2.79$ at $k = 12$ and $5.98$ at $k = 15$, and the CV crosses 1 in the same bracket. The ratio narrows again at $k = 21$ ($4.07$) not because the tail shortens but because the bulk has joined it: from $k = 18$ to $k = 21$ the median more than doubles ($269$ to $608\,\mathrm{s}$) while the 95th percentile grows $1.6\times$. A $k = 21$ job needs $23.7\,\mathrm{s}$ of

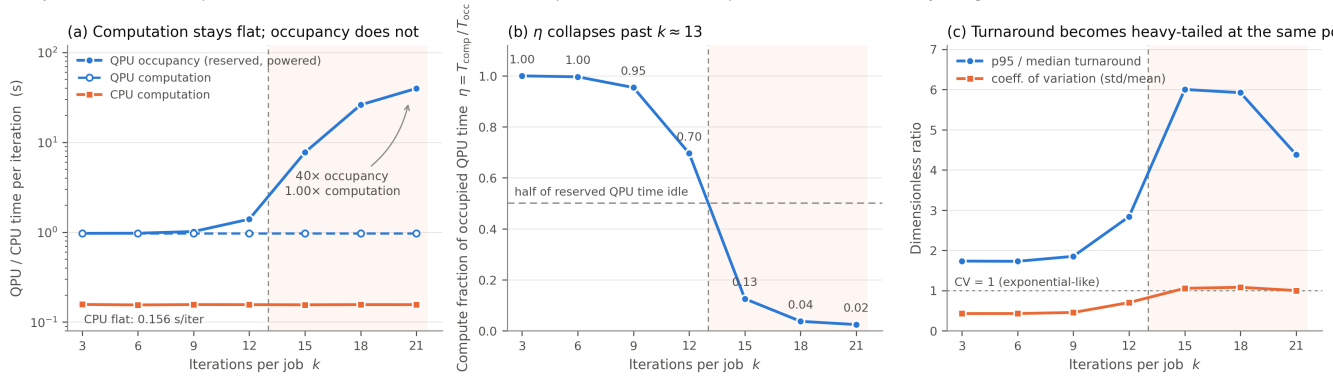



Fig. 5. Iteration scaling of a fixed 3000 job variational workload on a two QPU, two CPU fleet; only $k$ varies across the seven runs. (a) QPU time per iteration: computation is constant at 0.971 s while occupancy rises 40×; the classical phase is flat at 0.155 s to within 1%. (b) Compute fraction of occupied QPU time, $\eta$ of Eq. (14); it falls through one half near $k \approx 13$ and reaches 0.025 at $k = 21$. (c) The coefficient of variation crosses unity and the $p_{95}$/median ratio more than triples in the same bracket. The shaded band marks $\eta < \frac{1}{2}$.

computation and waits a median of 608 s for it. Two quantities from different columns of the record change character in the same interval: the evidence that one mechanism is responsible.

Because computation is fixed, the collapse must originate in re-acquisition: a job releases its qubit region at the end of every QPU phase and re-acquires one at the next. Raising $k$ does not raise the work per iteration; it raises how long the job *resides* in the system, and by Little's law a $k$ fold longer residency puts $k$ fold more jobs on the fleet at once. Concurrency demand reaches $\rho = 1$ only at $k \approx 16.6$ (Table VI), yet the compute fraction halves at $\rho \approx 0.78$: the knee falls while a fifth of the nominal qubit capacity is still formally free. Instrumenting the allocator shows what that gap is made of. Each blocked acquisition attempt (one second inside the phase) was classified by whether the device still held enough *free* qubits for the request. At the onset of the knee, $k = 12$, it did in 70.4% of 15,155 blocked attempts: on average 26.1 qubits were free against 18.1 requested, but the largest *connected* free region held only 12.4. This is connectivity fragmentation, a failure mode with no classical analogue: a heavy hex lattice that is 20% free can be unable to seat an 18 qubit circuit. As load deepens the balance shifts to plain capacity exhaustion, fragmentation accounting for 35.4% of 307,709 blocked attempts at $k = 15$ and 13.9% of 2.24 million at $k = 21$. Fragmentation initiates the collapse; saturation sustains it.

A superconducting QPU draws its full cryogenic baseline whenever it hosts a job, so the fleet power of Eq. (3) is paid for occupancy, computing or not. Per job energy is charged for computation only and is therefore linear in $k$: \$0.0096 per job at $k = 3$ and \$0.067 at $k = 21$, or \$29 and \$202 for the batch. Billing the occupied window at the same power would instead charge a $k = 21$ job about \$2.68, roughly $1/\eta \approx 40\times$ more, and the batch about \$8,000. The difference is the cost of reserved capacity that performed no work, which under a computation based attribution falls on the facility rather than the tenant.

### E. Implications

Increasing the iteration count of a hybrid job does not make it more expensive to compute; past the knee it makes it far more expensive to *hold*, and the holding dominates both the device side cost and the turnaround tail. Iteration count is therefore a *capacity planning* parameter with a threshold, not a workload parameter to be optimized in isolation. An operator sizing a fleet from single job measurements at small $k$ will extrapolate linearly and under provision QPU occupancy forty fold at $k = 21$, and an algorithm designer trading a longer loop for a shallower circuit may make a strictly worse system level trade even when per circuit metrics improve.

## VIII. Conclusion

In this work, we introduce a system-level energy and cost accounting framework for hybrid cloud computing. We demonstrate that job energy and cost can be systematically attributed through simulation by modeling workloads as alternating sequences of QPU and CPU execution segments, attributing consumption to execution time and macro-level device power states without requiring fine-grained hardware power models. While specific values depend on the chosen system configurations, workload patterns, and power accounting models, the framework provides a practical foundation for full-system quantum resource evaluation.

## Code and Data Availability

The source code and integrated data used for the simulations and experiments in this study are publicly accessible [40].


## Acknowledgment

Claude Code (Anthropic) was used to assist with proofreading and enhancing the writing quality and flow of this paper. The authors reviewed, edited, and take full responsibility for the final content.

## REFERENCES


[1] R. P. Feynman, "Quantum mechanical computers," 1986, vol. 16, no. 6, pp. 507–532.

[2] K. G. Johnson, S. Mirchandaney, E. Hoag, A. Heirich, A. Aiken, and T. J. Martínez, "Multinode multi-gpu two-electron integrals: Code generation using the regent language," *Journal of Chemical Theory and Computation*, vol. 18, no. 11, pp. 6522–6536, 2022.

[3] J. Ruane, E. Kiesow, J. Galatsanos, C. Dukatz, E. Blomquist, and P. Shukla, "Quantum index report 2025," *arXiv preprint arXiv:2506.04259*, 2025.

[4] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'brien, "A Variational Eigenvalue Solver on a Photonic Quantum Processor," *Nature communications*, vol. 5, no. 1, p. 4213, 2014.

[5] O. Ezratty, "Understanding Quantum Technologies 2023," *arXiv preprint arXiv:2111.15352*, 2021.

[6] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th ed. Cambridge, UK: Cambridge University Press, 2010.

[7] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. De Rose, and R. Buyya, "CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms," *Software: Practice and experience*, vol. 41, no. 1, pp. 23–50, 2011.

[8] A. Núñez, J. L. Vázquez-Poletti, A. C. Caminero, G. G. Castañe, J. Carretero, and I. M. Llorente, "iCanCloud: A Flexible and Scalable Cloud Infrastructure Simulator," in *Proceedings of the 2012 IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, 2012, pp. 506–513.

[9] H. Casanova, A. Legrand, and M. Quinson, "Simgrid: A Generic Framework for Large-scale Distributed Experiments," in *Tenth International Conference on Computer Modeling and Simulation (uksim 2008)*. IEEE, 2008, pp. 126–131.

[10] S. N. A. Jawaddi and A. Ismail, "Integrating OpenAI Gym and CloudSim Plus: A Simulation Environment for DRL Agent Training in Energy-driven Cloud Scaling," *Simulation Modelling Practice and Theory*, vol. 130, p. 102858, jan 2024.

[11] W. Chen and E. Deelman, "WorkflowSim: A Toolkit for Simulating Scientific Workflows in Distributed Environments," *2012 IEEE International Conference on E-Science*, pp. 1–8, 2012.

[12] D. Kliazovich, P. Bouvry, and S. U. Khan, "GreenCloud: A Packet-Level Simulator of Energy-Aware Cloud Computing Data Centers," *The Journal of Supercomputing*, vol. 62, no. 3, pp. 1263–1283, 2012.

[13] G. G. Castañe, J. L. Vázquez-Poletti, A. Núñez, and I. M. Llorente, "E-mc$^2$: A Formal Framework for Energy Modeling in Cloud Computing," *Simulation Modelling Practice and Theory*, vol. 39, pp. 56–75, 2013.

[14] A. T. Makaratzis and P. E. G. Sarigiannidis, "Energy Modeling in Cloud Simulation Frameworks: A Survey," *Simulation Modelling Practice and Theory*, vol. 83, pp. 119–134, 2018.

[15] H. Nguyen, B. Pham, and T. Tran, "iQuantum: A Discrete-Event Simulation Toolkit for Quantum Computing Infrastructures," *Software: Practice and Experience*, 2024, early Access.

[16] H. T. Nguyen, M. Usman, and R. Buyya, "QSimPy: A learning-centric simulation framework for quantum cloud resource management," in *Quantum Computing*. Elsevier, 2025, pp. 165–183.

[17] ——, "DRLQ: A Deep Reinforcement Learning-based Task Placement for Quantum Cloud Computing," in *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. IEEE, 2024, pp. 475–481.

[18] W. Luo, B. Baheri, T. Humble, J. Zhao, T. Zhan, R. Maharjan, and Q. Guan, "A Digital Twin of Scalable Quantum Clouds," in *39th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2025, pp. 165–175.

[19] T. Jones, A. Brown, I. Bush, and S. C. Benjamin, "QuEST and high performance simulation of quantum computers," *Sci Rep*, vol. 9, no. 1, p. 10736, 2019.

[20] J. Brennan, L. J. O'Riordan, K. G. Hanley, M. Doyle, M. Allalen, D. Brayford, L. Iapichino, and N. Moran, "QXTools: A Julia framework for distributed quantum circuit simulation," *J. Open Source Softw.*, vol. 7, p. 3711, 2022.

[21] S. Diadamo, J. Notzel, B. Zanger, and M. M. Bese, "Qunetsim: A software framework for quantum networks," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–12, 2021.

[22] H. Bian, J. Huang, J. Tang, R. Dong, L. Wu, and X. Wang, "PAS: A new powerful and simple quantum computing simulator," *Software: Practice and Experience*, vol. 53, no. 1, pp. 142–159, 2023.

[23] Y. Fan and Z. Lan, "DRAS-CQSim: A Reinforcement Learning Based Framework for HPC Cluster Scheduling," *Software Impacts*, vol. 8, p. 100077, may 2021.

[24] P. Senapati, Z. Wang, W. Jiang, T. S. Humble, B. Fang, S. Xu, and Q. Guan, "Towards redefining the reproducibility in quantum computing: A data analysis approach on NISQ devices," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2023, pp. 468–474.

[25] P. Senapati, T. M. Athawale, D. Pugmire, and Q. Guan, "Advancing comprehension of quantum application outputs: A visualization technique," in *Proceedings of the 2023 International Workshop on Quantum Classical Cooperative Computing (QCCC)*. Association for Computing Machinery, 2023, pp. 25–28.

[26] P. Senapati, Q. Guan, D. Pugmire, C. C. Lu, and T. M. Athawale, "Visualization of noisy and less noisy computational basis states in quantum computing," in *2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, 2025, pp. 12–21.

[27] P. Senapati, S. Y.-C. Chen, B. Fang, T. M. Athawale, A. Li, W. Jiang, C. C. Lu, and Q. Guan, "PQML: Enabling the predictive reproducibility on NISQ machines for quantum ML applications," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 2024, pp. 1413–1424.

[28] P. Senapati, S. Zhu, B. Peng, B. Fang, and Q. Guan, "UQ-VarQA: Benchmarking and characterizing NISQ computers through uncertainty quantification of variational quantum algorithms," in *43rd IEEE International Conference on Computer Design (ICCD)*. IEEE, 2025, pp. 356–363.

[29] P. Senapati, W. Luo, B. Peng, and Q. Guan, "Uncertainty quantification driven benchmarking and characterization of noisy quantum backends: A VQE case study," in *Proceedings of the Great Lakes Symposium on VLSI 2026 (GLSVLSI 2026)*. Association for Computing Machinery, 2026, pp. 830–835.

[30] V. Abraham, P. Senapati, H. Pathak, and B. Peng, "Elucidating many-body effects in molecular core spectra through real-time approaches: Efficient classical approximations and a quantum perspective," *The Journal of Chemical Physics*, vol. 164, no. 10, p. 104113, 2026.

[31] R. Zeng, P. Senapati, R. Tang, D. Liu, and Q. Guan, "Q-Bridge: Code translation for quantum machine learning via LLMs," 2026.

[32] P. Senapati, V. Abraham, Q. Guan, and B. Peng, "Unified uncertainty quantification framework bridging noisy quantum backends across variational quantum algorithms and quantum signal processing," 2026.

[33] K. Veigas *et al.*, "Towards energy efficiency of hpc data centers," *Electronics*, vol. 14, p. 3170, 2025.

[34] International Energy Agency, "Energy demand from ai," 2025. [Online]. Available: https://www.iea.org/reports/energy-and-ai/energy-demand-from-ai

[35] M. J. Martin, C. Hughes, G. Moreno, E. B. Jones, D. Sickinger, S. Narumanchi, and R. Grout, "Energy use in quantum data centers: Scaling the impact of computer architecture, qubit performance, size, and thermal parameters," *IEEE Transactions on Sustainable Computing*, vol. 7, no. 4, pp. 864–874, 2022.

[36] D. Jaschke and S. Montangero, "Is quantum computing green? an estimate for an energy-efficiency quantum advantage," *Quantum Science and Technology*, vol. 8, no. 2, p. 025001, 2023.

[37] A. Beloglazov and R. Buyya, "Optimal Online Deterministic Algorithms and Adaptive Heuristics for Energy and Performance Efficient Dynamic Consolidation of Virtual Machines in Cloud Data Centers," *Concurrency and Computation: Practice and Experience*, vol. 24, no. 13, pp. 1397–1420, 2012.

[38] Y. Matsumoto, T. Tanimoto, M. Tanaka, and T. Ono, "Low-power half-flux-quantum based counter circuits for cryogenic quantum computers," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2024, pp. 1007–1013.

[39] Y. Ueno, S. Imamura, Y. Tomida, T. Tanimoto, M. Tanaka, Y. Tabuchi, K. Inoue, and H. Nakamura, "C3-vqa: Cryogenic counter-based co-processor for variational quantum algorithms," *IEEE Transactions on Quantum Engineering*, vol. 6, pp. 1–17, 2025.

[40] W. Luo, "Source Code for System Level Energy and Cost Characterization of Hybrid Quantum–Classical Cloud Workloads," https://github.com/wolfatthegate/HybridCloudSim-2026.git, September 2026.